Курс «Базовая обработка данных на языке Python»

автор: Киреев В.С., к.т.н., доцент

Лабораторная работа № 3

Тема: «Базовые структуры данных и библиотека NumPy для обработки больших данных в Python»

1. Цель работы

1. Освоить базовые структуры данных библиотеки **NumPy**: ndarray, dtype, shape, strides.
2. Изучить **векторизацию** и **бродкастинг** — ключевые механизмы производительности NumPy.
3. Применить NumPy к решению прикладных задач: рекомендательные системы, поиск ближайших векторов, сжатие изображений методом главных компонент (PCA).
4. Научиться сравнивать эффективность NumPy и «чистого» Python по скорости и памяти.

Теоретическая часть

1. NumPy: назначение и возможности

NumPy (Numerical Python) — фундамент научных вычислений в Python. Библиотека предоставляет:

- **n-мерные массивы** (ndarray) — плотные блоки однородных данных;
- **векторизованные операции** — арифметика, математические функции, статистика;
- **линейную алгебру** (np.linalg): умножение матриц, обращение, собственные значения, SVD;
- **преобразование Фурье** (np.fft);
- **генерацию случайных чисел** (np.random).

Ключевое преимущество: NumPy сочетает гибкость Python со скоростью хорошо оптимизированного C-кода. Числовые операции выполняются над массивами **целиком**, а не поэлементно в цикле — это ускоряет код в 10–200 раз.

2. Массив ndarray: структура и свойства

Массив NumPy — это **однородный** контейнер фиксированного размера. В памяти он хранится как:

Поле	Что хранит
PyObject_HEAD	служебный заголовок Python-объекта
data	указатель на непрерывный блок данных
dimensions	размеры по осям (shape)
strides	шаги в байтах для перехода по каждой оси

Отличие от list**:**

Критерий	list	ndarray
Тип элементов	разнородные	однородные (dtype)
Память	28+ байт на объект + указатель	плотный блок фиксированного размера
Скорость арифметики	цикл на Python	векторизовано на C
Размерность	вложенные списки	до N-мерных массивов

Основные свойства массива:

```
import numpy as np
x = np.arange(12, dtype=np.int32).reshape(3, 4)
x.shape      # (3, 4)
x.ndim       # 2
x.size       # 12
x.dtype      # int32
x.itemsize   # 4 (байт на элемент)
x.nbytes     # 48 (всего байт)
x.strides    # (16, 4)
```

Важно: strides — это «сколько байт нужно пройти, чтобы перейти к следующему элементу по данной оси». Понимание strides критично для эффективной работы с массивами и для операций типа .T (транспонирование), которые не копируют данные, а лишь меняют strides.

3. Создание массивов

```
np.array([1, 2, 3])                 # из списка
np.zeros((3, 4))                    # нули
np.ones((2, 2), dtype=int)          # единицы
np.full((2, 3), 7)                  # заполнение константой
np.arange(0, 10, 2)                 # как range
np.linspace(0, 1, 5)                # равномерные точки
np.eye(4)                           # единичная матрица
np.random.rand(3, 3)                # равномерное [0,1)
np.random.randn(3, 3)               # нормальное N(0,1)
np.random.randint(1, 11, size=(3, 4))   # целые
```

4. Векторизация и бродкастинг

Векторизация — замена явных циклов операциями над массивами целиком:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = a + b          # array([5, 7, 9])
d = a * 2          # array([2, 4, 6])
```

Бродкастинг (broadcasting) — автоматическое расширение размерностей массивов разных форм при выполнении операций, **если они совместимы**.

Правила:

1. Формы выравниваются по последней оси.
2. Ось, где размер = 1, «растягивается» до размера другой.
3. Иначе — ValueError.

```
m = np.zeros((3, 4))
v = np.array([1, 2, 3, 4])
m + v          # v растягивается по строкам


col = np.array([[1], [2], [3]])    # (3,1)
row = np.array([10, 20, 30])       # (3,)
col + row                  # (3,3) — внешнее суммирование
```

5. Индексация, срезы, маски

```
a = np.array([10, 20, 30, 40, 50])
a[0]          # 10
```

```
a[-1]        # 50
a[1:4]       # [20, 30, 40]
a[::-1]      # разворот
a[a > 25]    # [30, 40, 50] — булева маска


m = np.arange(12).reshape(3, 4)
m[1, :]      # вторая строка
m[:, 2]      # третий столбец
m[1, 2]      # элемент
m[m % 2 == 0]  # только чётные
```

Fancy indexing — выборка по массиву индексов:

```
m[[0, 2], :]       # 1-я и 3-я строки
m[np.array([0, 1]), np.array([2, 3])]   # элементы (0,2) и (1,3)
```

6. Векторные операции

```
a = np.array([1., 2., 3.])
b = np.array([4., 5., 6.])


np.dot(a, b)              # скалярное произведение = 32
np.linalg.norm(a)         # ||a|| = √14 ≈ 3.742
np.linalg.norm(a - b)     # расстояние Евклида
cos = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
angle = np.degrees(np.arccos(np.clip(cos, -1, 1)))
```

Косинусная близость — ключевая метрика для поиска похожих векторов (тексты, эмбеддинги, рекомендации):

$$\text{sim}(q, v) = \frac{q \cdot v}{\| q \| \ \| v \|}, \ d_{\cos} = 1 - \text{sim}$$

Трюк: если все векторы заранее L2-нормированы, то косинусная близость = обычное скалярное произведение. Тогда Top-K для Qзапросов против базы Vразмера $N \times d$считается как **одно матричное умножение**:

$$S = \hat{Q} \cdot \hat{V}^T \in \mathbb{R}^{Q \times N}$$

7. Матричные операции и линейная алгебра

python

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A @ B              # матричное умножение
A.T                # транспонирование
np.linalg.inv(A)   # обратная матрица
np.linalg.det(A)   # определитель
np.trace(A)        # след
np.linalg.matrix_rank(A)
np.linalg.solve(A, b) # решение СЛАУ Ax = b
```

Разложение по собственным значениям (симметричной матрицы):

```
eigvals, eigvecs = np.linalg.eigh(A)  # для симметричных — устойчивее и быстрее
```

SVD (для произвольных матриц):

```
U, S, Vt = np.linalg.svd(X, full_matrices=False)
```

8. Метод главных компонент (PCA)

Идея. Найти ортогональные направления максимальной дисперсии в данных и спроецировать данные на первые kиз них — так мы переходим в пространство меньшей размерности, теряя минимум информации.

Шаги (реализация через NumPy):

1. Центрировать данные: $X_c = X - \bar{X}$(вычесть среднее по столбцам).
2. Построить ковариационную матрицу: $C = \frac{1}{n-1} X_c^T X_c$.
3. Найти собственные значения/векторы: np.linalg.eigh(C).
4. Отсортировать по убыванию собственных значений.
5. Спроецировать: $Z = X_c \cdot V_k$, где V_k— первые kсобственных векторов.
6. Оценить **долю объяснённой дисперсии**: $\lambda_i / \sum_j \lambda_j$.

Обратное преобразование (восстановление):

$$\hat{X} = Z \cdot V_k^T + \bar{X}$$

Применение к изображениям. Изображение — это ndarray формы (H, W, 3). PCA позволяет:

- Сжать изображение, сохранив лишь kглавных компонент из $\min(H, W)$;
- Оценить ошибку восстановления;

- Найти оптимальное *k*по scree-plot.

Схема сжатия для одного канала:

1. Разложить канал как матрицу X (H × W).
2. Вычесть среднее по столбцам.
3. Найти SVD: U, S, Vt = np.linalg.svd(X_c).
4. Оставить только первые *k*сингулярных чисел: X_rec = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :] + mean.
5. Ошибка = ||X - X_rec|| / ||X||.

9. Простейшая векторизация текстов

Для задач рекомендаций и поиска ближайших текстов полезно уметь превращать текст в числовой вектор. Самые простые способы — **Bag-of-Words (BoW)** и **бинарный BoW**:

```
docs = ["кот ест рыбу", "рыба ест кота", "пёс спит"]
vocab = sorted(set(" ".join(docs).split()))
# ["ест", "кот", "кота", "пёс", "рыбу", "рыба", "спит"]
def bow(doc, vocab):
    words = doc.split()
    return np.array([words.count(w) for w in vocab], dtype=float)
X = np.array([bow(d, vocab) for d in docs])
```

Далее — нормировка строк и косинусная близость между документами:

python

```
Xn = X / np.linalg.norm(X, axis=1, keepdims=True)
sim_matrix = Xn @ Xn.T          # матрица попарной близости документов
```

10. Производительность: NumPy vs list

Реальные бенчмарки показывают:

Операция	Список Python	NumPy	Ускорение
Сложение 1 млн int	~50 мс	~1 мс	×50
Скалярное произведение 1 млн	~30 мс	~0.5 мс	×60
Память (1 млн int)	~36 МБ	~8 МБ	×4.5

Правила производительности:

1. Заменяйте циклы векторными операциями.
2. Используйте float32 вместо float64, если точности достаточно.
3. Для Top-K используйте np.argpartition, а не np.sort.
4. Для пакетных запросов — матричное умножение (Q, d) @ (d, N).
5. Для баз > 10 млн векторов — приближённые методы (FAISS, HNSW).

Задания

В каждом задании используйте фиксированный random.seed(...) для воспроизводимости. Все ячейки снабжайте комментариями.

Курс «Базовая обработка данных на языке Python»

автор: Киреев В.С., к.т.н., доцент

Часть I. Простые операции с массивами (задания 1–5)

Задание 1. Создание и арифметика массивов

1. Создайте три массива V1, V2, V3 типа int размерности **10** каждый. Значения — целые от 1 до 10, число уникальных значений 4–9 (используйте np.random.choice с replace=True).
2. Объедините их вертикально в вектор V (30 элементов).
3. Создайте матрицу M размерности **30 × 30** типа int тем же способом.
4. Сложите поэлементно V1 и V2.
5. Сравните V1 и V2 и выведите индексы несовпадающих элементов.

Задание 2. Индексация и срезы

Для матрицы M из задания 1:

1. Выведите значение ячейки из последней строки и последнего столбца.
2. Извлеките подматрицу: строки 10–20, столбцы 5–6.
3. Найдите второе по убыванию значение всей матрицы.
4. Извлеките главную диагональ (np.diag).
5. Создайте булеву маску M > 5 и выведите количество таких элементов и их сумму.

Задание 3. Манипуляции с массивами

Для V и M из задания 1:

1. Измените тип данных V на строковый (.astype(str)).
2. Обратите массив V (первое значение — последним).
3. Сделайте матрицу M плоским массивом (.flatten()).
4. Переделайте вектор V формы (30,) в матрицу формы **5 × 6**.
5. Удалите из V все элементы, меньшие 2 (используйте булеву маску).

Задание 4. Статистика и визуализация

Для V и M:

1. Рассчитайте mean, median, var, std по вектору V.
2. То же — по матрице M (по осям 0 и 1).
3. Постройте гистограмму V через matplotlib.pyplot.hist с 10 бинами.
4. Рассчитайте матрицу корреляции для M: np.corrcoef(M, rowvar=False), визуализируйте её как тепловую карту (plt.imshow).
5. Создайте два случайных вектора по 10 элементов с 3 уникальными значениями и проверьте гипотезу об их независимости через scipy.stats.chi2_contingency.

Курс «Базовая обработка данных на языке Python»

автор: Киреев В.С., к.т.н., доцент

Задание 5. Векторная и матричная алгебра

Для M (30×30) и V (30,):

1. Умножьте матрицу M на вектор V (M @ V).
2. Найдите определитель M (np.linalg.det).
3. Найдите скалярное произведение V и транспонированного V.
4. Решите систему M @ x = V через np.linalg.solve. Проверьте ||M @ x − V|| ≈ 0.
5. Найдите собственные значения и векторы M (np.linalg.eig). Отсортируйте собственные значения по убыванию модуля.

Курс «Базовая обработка данных на языке Python»

автор: Киреев В.С., к.т.н., доцент

Часть II. Прикладные задачи (задания 6–20)

Задание 6. Бенчмарк: NumPy vs list

1. Сгенерируйте N = 1_000_000 случайных целых.
2. Реализуйте **поэлементное сложение** двух массивов двумя способами: через цикл Python (zip) и через numpy + numpy. Замерьте время через time.perf_counter().
3. То же — для **скалярного произведения**: цикл vs np.dot.
4. Измерьте размер в памяти для list (sys.getsizeof) и для ndarray (.nbytes). Сделайте вывод об экономии памяти.
5. Постройте графики: loglog-график времени (N от 10^3 до 10^7) и barplot памяти.

Задание 7. Бродкастинг

1. Сгенерируйте матрицу X формы (1000, 5) из нормального распределения с разными средними по столбцам.
2. Выполните **стандартизацию по столбцам** без циклов: из каждого столбца вычтите среднее и разделите на std (axis=0, keepdims=True).
3. Проверьте, что после стандартизации среднее по столбцам ≈ 0, а std ≈ 1.
4. Постройте внешнее произведение вектора-столбца (5, 1) и вектора-строки (1, 4) — покажите, что получается (5, 4).
5. Воспроизведите ошибку ValueError при несовместимых формах (2, 3) + (2, 4).

Задание 8. Собственные значения: анализ устойчивости

1. Сгенерируйте симметричную матрицу A формы (10, 10) как B + B.T.
2. Вычислите её собственные значения двумя способами: np.linalg.eig и np.linalg.eigh.
3. Убедитесь, что eigh даёт действительные значения и сортирует их по возрастанию.
4. Проверьте для первых двух векторов A @ v ≈ λ * v.
5. Исследуйте, как меняется спектр при добавлении к A небольшого шума ε * randn.

Задание 9. Сингулярное разложение (SVD)

1. Сгенерируйте матрицу A формы (20, 10).
2. Разложите её: U, S, Vt = np.linalg.svd(A, full_matrices=False).
3. Восстановите матрицу из полного разложения и убедитесь в точности.

4. Оставьте только первые 3 сингулярных числа и восстановите A_rec_3. Оцените ошибку.
5. Постройте scree-plot сингулярных чисел (S) и определите, сколько компонент описывают 95% энергии.

Задание 10. PCA на синтетических данных

1. Сгенерируйте данные: два скрытых фактора t1 ~ N(0, 3), t2 ~ N(0, 1) размером 500, смешанные случайной матрицей (4, 2) + шум.
2. Выполните PCA «руками» на NumPy: центрирование → Xc.T @ Xc / (n-1) → eigh.
3. Отсортируйте собственные значения по убыванию, выведите доли объяснённой дисперсии.
4. Спроецируйте данные на первые 2 компоненты, визуализируйте scatter-plot.
5. Восстановите данные из 2 компонент, оцените относительную ошибку.
6. Постройте scree-plot с кумулятивной долей.

Задание 11. Системы линейных уравнений

1. Сгенерируйте матрицу A (5, 5) с хорошей обусловленностью (например, диагонально доминирующую).
2. Сгенерируйте вектор b (5,).
3. Решите A x = b через np.linalg.solve.
4. Сравните с решением через np.linalg.pinv(A) @ b.
5. Вычислите число обусловленности np.linalg.cond(A). Оцените, насколько оно влияет на устойчивость решения.

Задание 12. Рекомендации: item-item коллаборативная фильтрация

1. Сгенерируйте синтетическую матрицу оценок R формы (200, 50): 0 = «нет оценки», значения 1–5 — оценки. Заполненность ~10%.
2. Рассчитайте косинусную близость между **товарами** (по столбцам): нормируйте столбцы и посчитайте R_norm.T @ R_norm. Обнулите диагональ.
3. Для товара #0 выведите топ-5 самых похожих товаров.
4. Реализуйте функцию recommend(user_id, k=5): считает scores = item_sim.T @ R[user_id], маскирует уже оценённое (-inf) и возвращает Top-K.
5. Визуализируйте для одного пользователя его оценки и рекомендации на одном barplot.

6. Сравните рекомендации для двух разных пользователей — насколько они пересекаются?

Задание 13. Рекомендации: user-user коллаборативная фильтрация

1. Используйте ту же матрицу R из задания 12.
2. Рассчитайте **косинусную близость между пользователями** (по строкам): R_norm @ R_norm.T.
3. Для пользователя #7 найдите топ-5 похожих пользователей.
4. Постройте рекомендации «по похожим пользователям»: возьмите товары, которые высоко оценили соседи, но не оценил сам пользователь. Ранжируйте по средней оценке с весами similarity.
5. Сравните результат с item-item подходом из задания 12. Какой метод даёт более осмысленные рекомендации для разреженных данных?

Задание 14. Поиск ближайших векторов (косинусное расстояние)

1. Сгенерируйте базу V из 50 000 случайных векторов размерности 64 (float32).
2. Сгенерируйте 100 запросов q.
3. Нормируйте V и q по L2. Вычислите матрицу близости S = qn @ Vn.T.
4. Найдите Top-10 соседей для каждого запроса с помощью np.argpartition. Внутри Top-10 отсортируйте по убыванию близости.
5. Для первого запроса выведите Top-10 с индексами и значениями близости.
6. Замерьте время выполнения:
   - полной сортировки np.argsort для поиска Top-10,
   - np.argpartition.
     Во сколько раз argpartition быстрее?

Задание 15. Кластеризация и поиск ближайших (K-means)

1. Сгенерируйте 3 «облака» точек в 2D: три центра + гауссов шум.
2. Реализуйте **K-means на NumPy** в 3 шага:
   - инициализация центроидов случайными точками;
   - присвоение: labels = argmin(||X[:, None] - centroids[None, :]||², axis=1);
   - обновление центроидов как среднего по кластеру.
3. Итерируйте до сходимости (метки не меняются).
4. Визуализируйте: точки по цветам кластеров + центроиды.

5. Для нового вектора найдите ближайший кластер по косинусной близости к центроиду.

Задание 16. Простая векторизация текстов: BoW

1. Сформируйте список из 10 коротких текстов (например, описания товаров, названия вакансий).
2. Постройте словарь (vocab) из уникальных слов.
3. Реализуйте функцию bow(doc, vocab) — вектор частот слов длиной len(vocab).
4. Постройте матрицу X формы (10, |vocab|) — «мешок слов».
5. Нормируйте строки и посчитайте матрицу косинусной близости между документами: Xn @ Xn.T.
6. Выведите топ-3 документа, наиболее близких к документу #0, с указанием близости.

Задание 17. Простая векторизация текстов: TF-IDF и поиск похожих

1. Используйте данные из задания 16.
2. Реализуйте **TF-IDF на NumPy**:
   - TF: частоты слов, нормированные на длину документа;
   - IDF: log(N / df) + 1, где df — число документов, где встречается слово;
   - tfidf = TF * IDF (бродкастинг по строкам).
3. Нормируйте строки и посчитайте матрицу косинусной близости.
4. Сравните с результатами обычного BoW из задания 16. Какие документы стали ближе, а какие — дальше? Объясните причину.

Задание 18. Поисковая система по мини-корпусу

1. Возьмите 30 коротких текстов (можно — описания вакансий или товаров).
2. Реализуйте **поиск по запросу**:
   - представьте запрос как TF-IDF-вектор, используя словарь из корпуса;
   - нормируйте его;
   - вычислите близость ко всем документам через скалярное произведение;
   - выведите Top-5 наиболее релевантных документов.
3. Реализуйте функцию search(query, top_k=5).
4. Проверьте работу на 3 разных запросах, покажите результаты.

5. Сравните ранжирование TF-IDF с ранжированием BoW для одного из запросов.

Задание 19. Матрица попарных расстояний и её оптимизация

1. Сгенерируйте 5 000 случайных векторов размерности 32.
2. Вычислите матрицу **попарных евклидовых расстояний** «в лоб»:

```
D = np.sqrt(((X[:, None, :] - X[None, :, :]) ** 2).sum(axis=-1))
```

3. Вычислите ту же матрицу через **трюк с формулой**:

```
X_sq = (X**2).sum(axis=1)
D2 = X_sq[:, None] + X_sq[None, :] - 2 * X @ X.T
D = np.sqrt(np.maximum(D2, 0))
```

4. Убедитесь, что результаты совпадают (np.allclose).
5. Замерьте время обоих подходов. Во сколько раз второй быстрее?
6. Оцените потребление памяти для матрицы D (5000×5000×8 байт). Почему для больших баз нельзя хранить всю матрицу в памяти?

Задание 20. Сжатие изображения методом главных компонент (PCA) и восстановление

Цель: взять реальное изображение, сжать его через PCA, восстановить и количественно оценить потерю качества.

1. **Загрузите изображение.** Скачайте любое изображение (например, с Википедии) через requests.get(...), преобразуйте в массив NumPy:

```
import requests, io
from PIL import Image
url = "https://upload.wikimedia.org/wikipedia/commons/thumb/..."
img = np.asarray(Image.open(io.BytesIO(requests.get(url).content)).convert("RGB"))
```

Если нет доступа к сети — используйте любое локальное изображение.

2. **Разложите** изображение на 3 канала R, G, B. Для одного канала (или для всей матрицы, если изображение приведено к оттенкам серого) выполните PCA:
   - центрирование: Xc = X - X.mean(axis=0);
   - SVD: U, S, Vt = np.linalg.svd(Xc, full_matrices=False).
3. **Сожмите, оставив** k **главных компонент** для k ∈ {5, 20, 50, 100, 200}:

```
X_rec = (U[:, :k] * S[:k]) @ Vt[:k, :] + X.mean(axis=0)
```

Убедитесь, что для **одного** k восстановление получается визуально близким.

4. **Оцените ошибки** для каждого k:

- относительная ошибка по норме Фробениуса: ||X − X_rec||_F / ||X||_F;
- PSNR (peak signal-to-noise ratio) — стандартная метрика для изображений.
  Постройте график: k vs ошибка и k vs PSNR.

5. **Оцените степень сжатия:** исходное изображение хранит $H \times W$ чисел. При оставлении k компонент хранится $k \cdot (H + W + 1)$ чисел. Постройте график «степень сжатия vs ошибка».
6. **Визуализация.** Покажите в одной фигуре:
   - оригинал;
   - восстановленное при k = 20;
   - восстановленное при k = 50;
   - восстановленное при k = 100;
   - карту разностей для одного из k (в оттенках серого).
7. **Объяснение.** Ответьте в комментариях:
   - какое k даёт «приемлемое» качество на ваш взгляд;
   - сколько данных удалось отбросить;
   - почему первые компоненты «отвечают» за крупные структуры, а последние — за шум.